

PROGRAMAÇÃO ORIENTADA A OBJETOS — CHECKPOINT 1

Nome: _____ Nota: _____

Orientações: atividade individual, sem consulta. Tempo: 50 minutos. Registre as respostas no quadro abaixo.

Q1a	Q1b	Q1c	Q1d	Q1e	Q1f	Q1g	Q2a
Q2b	Q2c	Q2d	Q2e	Q3	Q4	Q5	Q6

CENÁRIO: MEDIÇÃO DE CHUVA

Considere um sistema que registra a quantidade de chuva acumulada em diferentes locais. Cada objeto da classe MedidorChuva representa um medidor e armazena o total de chuva registrado até o momento.
Os arquivos abaixo pertencem ao mesmo projeto e ao mesmo pacote.

MedidorChuva.java	Main.java
<pre>public class MedidorChuva { String local; double totalMilimetros; void registrarChuva(double milimetros) { totalMilimetros = totalMilimetros + milimetros; } double consultarTotal() { return totalMilimetros; } }</pre>	<pre>public class Main { public static void main(String[] args) { MedidorChuva principal = new MedidorChuva(); principal.totalMilimetros = 6.0; MedidorChuva apoio = principal; MedidorChuva outro = new MedidorChuva(); outro.totalMilimetros = 6.0; System.out.println(principal == outro); // S1 System.out.println(principal == apoio); // S2 apoio = outro; System.out.println(principal == apoio); // S3 apoio.registrarChuva(3.0); System.out.println(principal.consultarTotal()); // S4 System.out.println(outro.consultarTotal()); // S5 apoio = principal; apoio.registrarChuva(5.0); System.out.println(principal.consultarTotal()); // S6 apoio.totalMilimetros = -4.0; principal.registrarChuva(6.0); System.out.println(principal.consultarTotal()); // S7 } }</pre>

Q01 [35 pontos]

Acompanhe, na ordem, a execução completa da classe Main e indique o valor exibido em cada uma das saídas identificadas de S1 a S7.

- a) [5 pontos]
Qual é a saída S1?
- b) [5 pontos]
Qual é a saída S2?
- c) [5 pontos]
Qual é a saída S3?
- d) [5 pontos]
Qual é a saída S4?
- e) [5 pontos]
Qual é a saída S5?
- f) [5 pontos]
Qual é a saída S6?
- g) [5 pontos]
Qual é a saída S7?

Q02 [25 pontos]

Analise as afirmativas abaixo e assinale (V) para as verdadeiras e (F) para as falsas.

- a) [5 pontos]
Na classe MedidorChuva, os atributos local e totalMilimetros representam comportamentos, enquanto os métodos registrarChuva e consultarTotal representam o estado do objeto.
- b) [5 pontos]
Depois que registrarChuva termina, o parâmetro milimetros permanece armazenado no objeto como se fosse um atributo.
- c) [5 pontos]
Como registrarChuva possui retorno void, ele não pode alterar os dados armazenados no objeto.
- d) [5 pontos]
Se uma variável receber o valor retornado por consultarTotal(), alterar essa variável depois também modifica totalMilimetros no objeto.
- e) [5 pontos]
Um método pode consultar ou alterar um atributo do próprio objeto sem receber esse atributo como parâmetro.

PROGRAMAÇÃO ORIENTADA A OBJETOS — CHECKPOINT 1

Nome: _____ Nota: _____

Q03 [10 pontos]

Agora queremos que cada MedidorChuva possa informar se o total de chuva acumulada atingiu um determinado limite.

Qual das propostas abaixo permite que o próprio objeto faça essa verificação usando o valor de totalMilímetros que ele já armazena?

A. Adicionar um atributo para guardar se o medidor está em alerta:

```
boolean emAlerta;
```

B. Fazer a comparação diretamente na classe Main:

```
boolean alerta = medidor.consultarTotal() >= limite;
```

C. Adicionar um método ao MedidorChuva, mas exigir que o total também seja informado:

```
boolean atingiuAlerta(double total, double limite) {
    return total >= limite;
}
```

D. Permitir que outros trechos do programa acessem diretamente o total para fazer a comparação:

```
public double totalMilímetros;
```

E. Adicionar um método que compara o limite recebido com o total armazenado no próprio objeto:

```
boolean atingiuAlerta(double limite) {
    return totalMilímetros >= limite;
}
```

Q04 [10 pontos]

Na versão inicial de MedidorChuva, o atributo totalMilímetros foi declarado sem um modificador de acesso:

```
double totalMilímetros;
```

Mesmo assim, a classe Main consegue executar:

```
principal.totalMilímetros = 6.0;
```

Considerando que Main e MedidorChuva pertencem ao mesmo pacote, qual alternativa explica corretamente por que esse acesso é permitido?

- A. Porque o método main possui permissão especial para acessar os atributos de qualquer classe.
- B. Porque um atributo declarado sem modificador de acesso pode ser acessado por outras classes do mesmo pacote.
- C. Porque todo atributo do tipo double pode ser acessado diretamente por qualquer classe do projeto.
- D. Porque uma variável que guarda a referência de um objeto pode acessar qualquer atributo desse objeto, independentemente do modificador de acesso.
- E. Porque a criação do objeto com new MedidorChuva() torna seus atributos públicos.

Q05 [10 pontos]

Na versão inicial de MedidorChuva, o atributo totalMilímetros pode ser alterado diretamente por qualquer código que tenha acesso ao objeto.

Para controlar melhor essas alterações, considere agora que:

- totalMilímetros é private;
- cada MedidorChuva começa com total 0.0;
- registrarChuva soma apenas valores maiores que zero;
- consultarTotal informa o total acumulado.

Considere o seguinte início:

```
MedidorChuva principal = new MedidorChuva();
MedidorChuva apoio = principal;
```

Qual alternativa mantém principal e apoio apontando para o mesmo objeto e, ao final, faz consultarTotal() devolver 12.0 pelas duas referências?

A.

```
principal.registrarChuva(7.0);
apoio = new MedidorChuva();
apoio.registrarChuva(5.0);
```

B.

```
principal.totalMilímetros = 7.0;
apoio.totalMilímetros = -3.0;
apoio.totalMilímetros = 12.0;
```

C.

```
principal.registrarChuva(7.0);
apoio.registrarChuva(3.0);
apoio.registrarChuva(5.0);
```

D.

```
principal.registrarChuva(7.0);
apoio.registrarChuva(-3.0);
apoio.registrarChuva(5.0);
```

E.

```
principal.registrarChuva(7.0);
apoio.consultarTotal();
apoio.registrarChuva(-3.0);
```

Q06 [10 pontos]

Agora totalMilímetros é private, e seu valor só deve ser alterado por meio de registrarChuva, que aceita apenas valores maiores que zero.

Considere a proposta de adicionar o seguinte método à classe:

```
public void definirTotal(double novoTotal) {
    totalMilímetros = novoTotal;
}
```

Essa mudança mantém o controle sobre as alterações de totalMilímetros?

- A. Sim. Como definirTotal pertence à própria classe, qualquer alteração realizada por ele respeita automaticamente as regras definidas para MedidorChuva.
- B. Não. Um método public não pode alterar um atributo private da própria classe.
- C. Não. O método definirTotal permite substituir diretamente o valor de totalMilímetros, sem passar pela validação feita em registrarChuva.
- D. Sim. Como totalMilímetros é private, nenhum método público pode atribuir diretamente um novo valor ao atributo.
- E. Não. Ao receber novoTotal como parâmetro, totalMilímetros deixa de ser private.